

MARMELO KITCHEN

Technical Requirements Document

Mobile App · Website · Admin Dashboard

FOR INTERNAL DEVELOPMENT USE ONLY

Version 1.0 | February 2026

1. Project Overview

1.1 Project Summary

Development of a complete digital platform for Marmelo Kitchen consisting of:

- iOS & Android mobile applications
- Responsive public-facing website
- Admin dashboard (web-based)
- RESTful API backend
- Database infrastructure

1.2 Business Context

Marmelo Kitchen is a London-based food business offering:

- General Store (in-person and online sales with in-store collection)
- Catering Services (event catering across London)
- Venue Hire (private event space)
- Events & Community Programming

Current State: Fragmented across Squarespace (website) and Shopify (store). Goal: Replace with unified owned platform.

1.3 Technical Stack Recommendations

Component	Recommended Technology	Justification
Mobile App	React Native	Single codebase for iOS/Android
Website Frontend	React.js or Next.js	Fast, SEO-friendly, component reuse
Admin Dashboard	React.js + Tailwind CSS	Rapid development, maintainable
Backend API	Node.js (Express) or Python (FastAPI)	RESTful API, easy integration
Database	PostgreSQL	Reliable, ACID compliant, scalable
File Storage	AWS S3 or Cloudinary	Product images, media assets
Payments	Stripe or Razorpay	Secure, widely supported
Push Notifications	Firebase Cloud Messaging	Cross-platform, free tier available
Hosting	AWS, DigitalOcean, or Railway	Scalable, cost-effective

Note to Development Team:

Stack is flexible — use what your team is most proficient in.
Priority: Clean architecture, maintainability, and documentation.

All code must be version-controlled (Git) with clear commit history.

1.4 Timeline & Milestones

Phase	Deliverables	Duration
Phase 1	Wireframes, UI mockups, database schema, API design doc	Weeks 1-2
Phase 2	Website + Admin Dashboard (functional)	Weeks 3-6
Phase 3	Mobile app (iOS + Android, connected to backend)	Weeks 5-10
Phase 4	QA testing, bug fixes, performance optimization	Weeks 11-12
Phase 5	Deployment, App Store submission, go-live	Weeks 13-14

2. Mobile App Specifications

2.1 Platform Requirements

- iOS: Minimum iOS 14.0+, target latest iOS version
- Android: Minimum Android 8.0 (API 26)+, target latest Android version
- Single codebase using React Native (or Flutter if team prefers)

2.2 Core Features

2.2.1 User Authentication & Onboarding

- Email/password signup and login
- Social login: Sign in with Google, Sign in with Apple (iOS)
- Email verification (send confirmation code)
- Password reset via email
- Profile creation: Name, email, phone number (optional), delivery address

Technical Details:

Use JWT tokens for session management.

Implement secure storage (Keychain on iOS, Keystore on Android) for tokens.

Password hashing: bcrypt or Argon2.

2.2.2 Product Browsing & Search

- Homepage: Featured products, categories, promotional banners
- Product listing: Grid or list view with images, name, price
- Search: Text search by product name, category
- Filters: Category, price range, availability
- Product detail page:
 - Product images (swipeable gallery)
 - Name, description, price, stock status
 - Add to basket button
 - Quantity selector

Technical Details:

API endpoint: GET /api/products (supports pagination, filters, search query)

Image lazy loading to optimize performance

Cache product listings locally to reduce API calls

2.2.3 Shopping Basket & Checkout

- Add items to basket (persist across sessions)
- View basket: List items, quantities, subtotal
- Update quantities, remove items
- Checkout flow:
 - Confirm items and quantities
 - Select collection date and time (calendar picker)
 - Payment: Integrate Stripe/Razorpay
 - Order confirmation screen

Technical Details:

Basket state: Store in local state + sync to backend for logged-in users

Payment: Stripe SDK or Razorpay SDK integration

API endpoint: POST /api/orders (create order)

Webhook: Listen for payment confirmation from payment gateway

2.2.4 Order Tracking

- Order history: List all past orders (newest first)
- Order detail view:

- Order ID, date, total amount
- Items purchased
- Order status: Confirmed, In Preparation, Ready to Collect, Collected
- Collection time
- Real-time status updates (via API polling or WebSockets)

Technical Details:

API endpoint: GET /api/orders (fetch user orders)

Status updates: Poll every 30 seconds or use WebSockets for real-time updates

Push notifications when status changes (see 2.2.7)

2.2.5 Catering Booking

- Catering page: Info about catering services, sample menus
- Booking form:
 - Event type (wedding, corporate, private party, other)
 - Event date, time, location
 - Number of guests
 - Menu preferences / dietary requirements (text input)
 - Contact information (pre-filled from profile)
 - Additional notes
- Submit enquiry → Admin receives notification
- Confirmation message: 'We'll be in touch within 24 hours'

Technical Details:

API endpoint: POST /api/catering/enquiries

Email notification to admin on submission

Store enquiry in database with status: Pending, Approved, Declined

2.2.6 Venue Hire Booking

- Venue hire page: Photos of space, capacity, pricing info
- Booking form:
 - Event type
 - Preferred date and time
 - Number of attendees
 - Setup requirements (text input)
 - Contact information
- Submit enquiry → Admin receives notification

Technical Details:

API endpoint: POST /api/venue/enquiries

Same structure as catering enquiries

2.2.7 Push Notifications

- Send notifications when:
 - Order status changes (e.g., 'Your order is ready to collect!')
 - New promotions or events
 - Catering/venue enquiry response from admin
- User can enable/disable notifications in settings

Technical Details:

Use Firebase Cloud Messaging (FCM) for both iOS and Android

Store FCM device tokens in database linked to user accounts

Admin dashboard can trigger manual push notifications

2.2.8 User Profile & Settings

- View and edit profile:
 - Name, email, phone number
 - Default delivery/collection address
- Change password
- Notification preferences
- Logout

2.2.9 Additional Screens

- About Marmelo: Static content about the business
- Contact Us: Phone, email, Google Maps integration showing store location
- Terms & Conditions, Privacy Policy (static pages, web view)

2.3 UI/UX Design Guidelines

- Brand colors: Align with Marmelo's existing brand identity (greens, earth tones)
- Typography: Clean, modern fonts (e.g., Inter, Nunito, or custom brand font)
- Imagery: High-quality product photos, lifestyle images
- Navigation: Bottom tab bar with Home, Shop, Orders, Profile
- Accessibility: Support dark mode, minimum touch target size 44x44pt, screen reader compatible

2.4 Performance & Optimization

- App launch time: < 2 seconds on mid-range devices
- Image optimization: Compress images, use WebP format where supported
- Offline support: Cache product data, allow browsing when offline (checkout requires internet)
- Error handling: Graceful error messages, retry mechanisms

3. Website Specifications

3.1 Platform & Hosting

- Responsive web application (mobile-first design)
- SEO optimized (meta tags, structured data, sitemap.xml)
- SSL certificate (HTTPS)
- Domain: marmelokitchen.com (or client-provided domain)

3.2 Pages & Features

3.2.1 Homepage

- Hero section: Large image/video, tagline, CTA button ('Shop Now' or 'Explore')
- Featured products carousel
- About section: Brief story of Marmelo Kitchen
- Services overview: Links to Catering, Venue Hire, Events
- Newsletter signup form (email collection)
- Footer: Links to all pages, social media icons, contact info

3.2.2 Online Store

- Product catalog: Grid layout, filters (category, price), search bar
- Product pages: Same as mobile app (images, description, price, add to basket)
- Shopping basket: Persistent, visible in header
- Checkout: Same flow as mobile app (collection time, payment)
- Guest checkout: Allow purchase without creating account (capture email for order confirmation)

3.2.3 Catering Page

- Service description, sample menus, testimonials
- Booking form: Same fields as mobile app
- Photo gallery of past catering events

3.2.4 Venue Hire Page

- Venue details: Capacity, amenities, pricing structure
- Photo gallery of the space

- Booking form: Same as mobile app
- FAQ section (common questions about venue hire)

3.2.5 Events / What's On Page

- Upcoming events calendar
- Event detail pages: Date, time, description, RSVP or ticket link
- Admin can add/edit events via dashboard

3.2.6 About Page

- Marmelo's story, mission, team photos
- Community involvement, sustainability practices

3.2.7 Contact Page

- Contact form: Name, email, subject, message
- Store address, phone, email
- Google Maps embed showing store location
- Opening hours

3.2.8 User Account Pages

- Login / Signup
- My Account: Order history, profile settings (same as mobile app)

3.3 Technical Requirements

- Responsive design: Optimized for mobile, tablet, desktop
- Page load time: < 3 seconds (Lighthouse score 90+)
- Browser support: Latest 2 versions of Chrome, Firefox, Safari, Edge
- Analytics: Google Analytics or similar integrated
- Cookie consent banner (GDPR compliance)

4. Admin Dashboard Specifications

4.1 Access & Authentication

- Secure login: Admin email/password, 2FA recommended
- Role-based access control (RBAC):
 - Super Admin: Full access
 - Staff: View orders, update order status (limited product editing)

4.2 Dashboard Home

- Summary widgets:
 - Today's revenue
 - Orders today (total count, breakdown by status)
 - Pending catering/venue enquiries
 - Low stock alerts
- Quick actions: Add new product, view recent orders

4.3 Order Management

- Orders table: List all orders (filter by date, status, customer)
- Order detail view:
 - Customer info, items, total, collection time
 - Update status dropdown: Confirmed, In Preparation, Ready to Collect, Collected
 - Send notification to customer on status change
- Export orders to CSV (for accounting)

4.4 Product Management

- Product list: Table with name, price, stock, category, status (active/inactive)
- Add new product:
 - Name, description, price, category, stock quantity
 - Upload images (multiple)
 - Active/inactive toggle
- Edit product: Update any field
- Delete product (soft delete recommended — archive instead of hard delete)
- Bulk actions: Activate/deactivate multiple products, update prices

4.5 Catering & Venue Enquiry Management

- Enquiries table: List all enquiries (filter by type, status, date)
- Enquiry detail view:
 - Customer info, event details, special requests
 - Status: Pending, Approved, Declined
 - Admin notes (internal, not visible to customer)
- Respond to enquiry: Send email directly from dashboard

4.6 Customer Management

- Customer list: View all registered users
- Customer detail: Order history, contact info, account status
- Search customers by name, email, phone

4.7 Events Management

- Add/edit/delete events
- Event form: Title, date/time, description, image, RSVP link
- Events display on public 'What's On' page

4.8 Settings & Configuration

- Store settings: Name, address, contact details, opening hours
- Payment gateway configuration (API keys for Stripe/Razorpay)
- Email templates: Customize order confirmation, enquiry responses
- Notification settings: Enable/disable customer push notifications

4.9 Reporting & Analytics

- Sales reports: Daily, weekly, monthly revenue
- Top-selling products
- Customer acquisition: New signups over time
- Export reports to PDF or CSV

5. Backend API Specifications

5.1 API Architecture

- RESTful API design
- JSON request/response format
- Authentication: JWT tokens (Bearer token in Authorization header)
- API versioning: /api/v1/...
- Rate limiting: 100 requests per minute per IP/user

5.2 Core Endpoints

Authentication

Method	Endpoint	Description
POST	/api/auth/register	User signup (email, password, name)
POST	/api/auth/login	User login, returns JWT token
POST	/api/auth/logout	Invalidate JWT token
POST	/api/auth/reset-password	Request password reset email

Products

Method	Endpoint	Description
GET	/api/products	List all products (supports filters, search, pagination)
GET	/api/products/:id	Get single product details
POST	/api/products	Create new product (admin only)
PUT	/api/products/:id	Update product (admin only)
DELETE	/api/products/:id	Delete product (admin only)

Orders

Method	Endpoint	Description
GET	/api/orders	List user's orders (customer) or all orders (admin)
GET	/api/orders/:id	Get order details
POST	/api/orders	Create new order (checkout)
PATCH	/api/orders/:id/status	Update order status (admin only)

Catering & Venue Enquiries

Method	Endpoint	Description
POST	/api/catering/enquiries	Submit catering enquiry
POST	/api/venue/enquiries	Submit venue hire enquiry
GET	/api/enquiries	List all enquiries (admin only)
PATCH	/api/enquiries/:id/status	Update enquiry status (admin only)

User Profile

Method	Endpoint	Description
GET	/api/profile	Get current user's profile
PUT	/api/profile	Update profile information

Push Notifications

Method	Endpoint	Description
--------	----------	-------------

POST	/api/notifications/register	Register device FCM token
POST	/api/notifications/send	Send push notification (admin only)

5.3 Database Schema (High-Level)

Users Table

- id (UUID, primary key)
- email (unique, indexed)
- password_hash
- name
- phone
- role (customer, staff, admin)
- created_at, updated_at

Products Table

- id (UUID, primary key)
- name
- description (text)
- price (decimal)
- stock_quantity (integer)
- category (string or foreign key to Categories table)
- images (JSON array of image URLs)
- is_active (boolean)
- created_at, updated_at

Orders Table

- id (UUID, primary key)
- user_id (foreign key to Users)
- total_amount (decimal)
- status (confirmed, in_preparation, ready_to_collect, collected)
- collection_time (datetime)
- payment_status (paid, pending, failed)
- payment_id (string, from Stripe/Razorpay)
- created_at, updated_at

Order Items Table

- id (UUID, primary key)
- order_id (foreign key to Orders)
- product_id (foreign key to Products)
- quantity (integer)

- price_at_purchase (decimal, snapshot of price at time of order)

Enquiries Table (Catering & Venue)

- id (UUID, primary key)
- user_id (foreign key to Users, optional if guest enquiry)
- type (catering, venue)
- event_type, event_date, event_time, location, guests, details (text)
- status (pending, approved, declined)
- admin_notes (text)
- created_at, updated_at

Events Table

- id (UUID, primary key)
- title, description, event_date, event_time
- image_url
- rsvp_link (optional)
- created_at, updated_at

6. Security & Compliance

6.1 Data Security

- All passwords hashed using bcrypt (min cost factor 10)
- HTTPS enforced (SSL/TLS certificate)
- Secure storage of API keys and secrets (environment variables, not hardcoded)
- SQL injection prevention: Use parameterized queries/ORM
- XSS protection: Sanitize all user inputs
- CSRF protection: CSRF tokens for state-changing requests

6.2 Payment Security

- PCI DSS compliance: Do NOT store card details — use Stripe/Razorpay tokenization
- Payment webhooks: Verify webhook signatures to prevent fraud

6.3 GDPR Compliance (if applicable)

- Cookie consent banner on website
- Privacy policy and terms of service pages
- User data export: Allow users to download their data
- Right to deletion: Implement user account deletion feature

6.4 Testing & Quality Assurance

- Unit tests: Critical backend logic (authentication, order processing)
- Integration tests: API endpoints
- End-to-end tests: User flows (signup, checkout, order tracking)
- Manual QA: Test on real devices (iOS, Android, multiple browsers)
- Performance testing: Load testing on API (handle 100 concurrent users)

7. Deployment & DevOps

7.1 Hosting & Infrastructure

- Backend API: Deploy on AWS EC2, DigitalOcean Droplet, or Railway
- Database: Managed PostgreSQL (AWS RDS, DigitalOcean Managed DB, or Supabase)
- File storage: AWS S3 or Cloudinary for product images
- CDN: CloudFlare or AWS CloudFront for static assets

7.2 CI/CD Pipeline

- Version control: Git (GitHub, GitLab, or Bitbucket)
- Automated deployment:
 - On push to main branch → run tests → deploy to production
 - On push to dev branch → deploy to staging environment
- Tools: GitHub Actions, GitLab CI, or CircleCI

7.3 Monitoring & Logging

- Application monitoring: Sentry or LogRocket for error tracking
- Server monitoring: Uptime checks (UptimeRobot, Pingdom)
- Logging: Centralized logs (Logtail, Papertrail, or ELK stack)
- Alerts: Email/Slack notifications for critical errors or downtime

7.4 Backups

- Database backups: Daily automated backups, retained for 30 days
- File backups: Versioning enabled on S3 buckets

8. Post-Launch Support & Maintenance

8.1 Immediate Post-Launch (1 Month)

- Bug fixes: Critical bugs fixed within 24 hours, non-critical within 1 week
- Performance monitoring: Daily checks, optimize if needed
- User feedback collection: Implement in-app feedback form or survey

8.2 Ongoing Maintenance (Optional Support Package)

- Monthly updates: Security patches, dependency updates
- Feature enhancements: Based on client roadmap
- Performance optimization: Quarterly reviews
- Analytics review: Monthly report on user behavior, sales trends

8.3 Documentation Handover

- Technical documentation:
 - API documentation (Swagger/OpenAPI spec)
 - Database schema diagram
 - Deployment guide (how to deploy updates)
- Admin user guide:
 - How to add products, update orders, manage enquiries
 - Screenshots and step-by-step instructions
- Developer README: Setup instructions for future developers

9. Appendix

9.1 Glossary

Term	Definition
API	Application Programming Interface — allows different software to communicate
JWT	JSON Web Token — secure way to transmit authentication data
FCM	Firebase Cloud Messaging — service for sending push notifications
RBAC	Role-Based Access Control — permissions based on user roles
REST	Representational State Transfer — architectural style for APIs

9.2 Reference Links

- React Native: <https://reactnative.dev>

- Stripe API: <https://stripe.com/docs/api>
- Firebase Cloud Messaging: <https://firebase.google.com/docs/cloud-messaging>
- PostgreSQL: <https://www.postgresql.org/docs/>